



FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE
INGENIERÍA DE SISTEMAS
Y COMPUTACIÓN



**BASE DE
DATOS II**

SEMANA 11_

DESARROLLO

INFOGRAFÍAS

DE APRENDIZAJE



DOCENTE : Ing. Fernández Raúl Enrique



ESTUDIANTE : Rosales Crispín Aristóteles O



CICLO : V | **CARRERA:** Ing.Sistemas y Computacion



AUTENTIFICACIÓN SQL Y WINDOWS: DIFERENCIAS Y PRÁCTICAS SEGURAS



Elegir y configurar correctamente el método de autenticación es clave para proteger los datos y controlar el acceso a SQL Server.



¿QUÉ ES LA AUTENTIFICACIÓN?

Es el proceso mediante el cual un usuario se verifica para obtener acceso a SQL Server y a sus recursos.



MÉTODOS DE AUTENTIFICACIÓN EN SQL SERVER

SQL Server permite dos modos principales:



Autenticación SQL
(Usuarios de SQL Server)



Autenticación Windows
(Usuarios de Windows/AD)

DIFERENCIAS CLAVE

CARACTERÍSTICA	AUTENTIFICACIÓN SQL	AUTENTIFICACIÓN WINDOWS
Origen de credenciales	Credenciales almacenadas en SQL Server (nombre de usuario y contraseña).	Credenciales de Windows o Active Directory (cuentas de dominio).
Gestión de usuarios	Se crean y administran dentro de SQL Server.	Se administran en Active Directory o localmente.
Integración con Active Directory	No requiere Active Directory.	Integración nativa con Active Directory.
Seguridad	Las contraseñas se almacenan en SQL Server (pueden ser más vulnerables si no se gestionan bien).	Más segura (usa políticas de contraseña, bloqueo de cuentas, Kerberos/NTLM).
Escalabilidad	Menos escalable en entornos grandes.	Altamente escalable en entornos empresariales.
Uso recomendado	Aplicaciones independientes, usuarios externos o cuando no se usa Active Directory.	Entornos corporativos con Active Directory, aplicaciones internas y servicios.

PRÁCTICAS SEGURAS – AUTENTIFICACIÓN SQL



1. USAR CONTRASEÑAS FUERTES

Mínimo 12 caracteres, combinando mayúsculas, minúsculas, números y símbolos.



2. APLICAR POLÍTICAS DE CONTRASEÑA

Habilitar la expiración, historial y complejidad de contraseñas.



3. LIMITAR PERMISOS

Otorgar solo los permisos necesarios para cada usuario (principio de mínimo privilegio).



4. CAMBIAR CONTRASEÑAS REGULARMENTE

Establecer un ciclo de cambio y no reutilizar contraseñas antiguas.



5. AUDITORÍA Y MONITOREO

Registrar intentos de inicio de sesión y revisar accesos sospechosos.



PRÁCTICAS SEGURAS – AUTENTIFICACIÓN WINDOWS



1. USAR GRUPOS DE SEGURIDAD

Asignar permisos a grupos de AD, no a usuarios individuales.



2. APLICAR POLÍTICAS DE GRUPO (GPO)

Enforce contraseñas fuertes, bloqueo de cuenta e historial desde GPO.



3. USAR KERBEROS (RECOMENDADO)

Proporciona autenticación mutua y evita ataques de suplantación.



4. REVISAR MEMBRESÍAS Y ACCESOS

Auditar periódicamente grupos y permisos asignados.



5. SINCRONIZACIÓN Y ACTUALIZACIONES

Mantener Active Directory y SQL Server actualizados y sincronizados.

¿CUÁNDO USAR CADA UNA?



USA AUTENTIFICACIÓN SQL CUANDO:

- No tienes Active Directory.
- Los usuarios son externos o independientes.
- Necesitas cuentas específicas para aplicaciones.



USA AUTENTIFICACIÓN WINDOWS CUANDO:

- Tu organización usa Active Directory.
- Necesitas administración centralizada.
- Requieres mayor seguridad y escalabilidad.

RECOMENDACIÓN GENERAL



En entornos empresariales, usa Autenticación Windows siempre que sea posible por su mayor seguridad, integración y facilidad de administración.

Para casos específicos, utiliza Autenticación SQL aplicando todas las prácticas seguras.

CONFIGURACIÓN RÁPIDA

HABILITAR AMBOS MODOS

En SQL Server Configuration Manager:
Security > SQL Server and Windows Authentication mode



BUENAS PRÁCTICAS GENERALES

- ✓ Mantener SQL Server actualizado.
- ✓ Respalidar configuraciones y cuentas.
- ✓ Aplicar cifrado (TDE, SSL/TLS).
- ✓ Auditar y monitorear accesos.



CUENTAS DE SERVICIO Y CONFIGURACIÓN DE SERVIDOR

Buenas prácticas para garantizar seguridad, estabilidad y rendimiento en SQL Server



¿QUÉ SON LAS CUENTAS DE SERVICIO?



Son cuentas de Windows que se utilizan para ejecutar los servicios de SQL Server y otros componentes del sistema.

Su correcta configuración mejora la seguridad, la estabilidad y facilita la administración.

SERVICIOS DE SQL SERVER Y SUS CUENTAS ASOCIADAS

Servicio	Descripción	Cuenta recomendada
 SQL Server (Database Engine)	Motor de base de datos. Procesa consultas y administra datos.	Cuenta de servicio dedicada (ej. DOMINIO\svc-sqlengine)
 SQL Server Agent	Ejecuta trabajos, alertas y operaciones programadas.	Cuenta de servicio dedicada (ej. DOMINIO\svc-sqlagent)
 SQL Server Browser	Permite a las aplicaciones encontrar instancias de SQL Server.	Cuenta de servicio dedicada o cuenta del sistema local
 SQL Server Integration Services (SSIS)	Ejecuta paquetes de SSIS.	Cuenta de servicio dedicada

TIPOS DE CUENTAS PARA SERVICIOS



Cuenta de servicio dedicada (Recomendada)

Cuenta específica para cada servicio. Mejora la seguridad y el control.



Cuenta de dominio

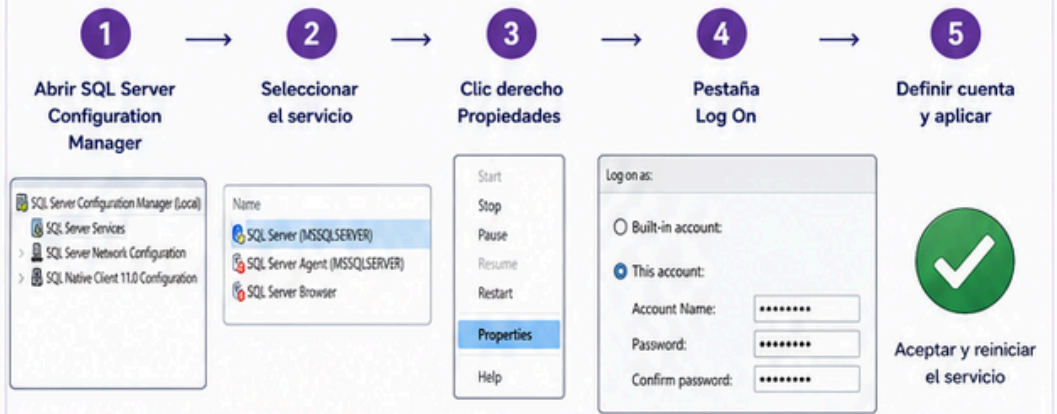
Útil para entornos donde los servicios necesitan acceder a recursos de red.



Cuenta del sistema local

Fácil de configurar, pero con menos control y permisos limitados.

CONFIGURACIÓN DE CUENTAS DE SERVICIO EN SQL SERVER



BUENAS PRÁCTICAS PARA CUENTAS DE SERVICIO



Usar cuentas dedicadas para cada servicio



Asignar solo los permisos necesarios (principio de menor privilegio)



Evitar usar cuentas de administrador de dominio



Configurar cambio periódico de contraseñas



Documentar las cuentas y permisos asignados



Monitorear y auditar el uso de cuentas de servicio

CONFIGURACIONES IMPORTANTES DEL SERVIDOR

Configuración	Recomendación
 Autenticación	Habilitar Autenticación Mixta solo si es necesario. Preferir Autenticación Windows.
 Firewall	Permitir solo los puertos y IP necesarias.
 Actualizaciones	Mantener el servidor y SQL Server actualizados.
 Copias de seguridad	Programar y verificar copias de seguridad periódicas.
 Auditoría	Habilitar auditoría para rastrear actividades importantes.
 Cifrado	Usar cifrado para datos en tránsito y en reposo.

EJEMPLO DE ASIGNACIÓN DE PERMISOS

Cuenta de servicio (DOMINIO\svc-sqlengine)  SQL Server	Permisos mínimos necesarios ✓ Iniciar sesión como servicio ✓ Leer como servicio ✓ Permisos en archivos y carpetas donde se almacenan los datos, logs y copias de seguridad
--	---

BENEFICIOS



Mayor seguridad
Reduce riesgos por uso de cuentas con privilegios elevados.



Mejor estabilidad
Aísla procesos y facilita la solución de problemas.



Administración eficiente
Facilita auditorías y control de accesos.



Rendimiento óptimo
Configuraciones adecuadas mejoran el rendimiento del servidor.

CREACIÓN DE ROLES FIJOS Y PERSONALIZADOS

Organiza permisos, simplifica la administración y fortalece la seguridad de tu base de datos.



¿QUÉ ES UN ROL EN SQL SERVER?



Un rol es una colección de permisos que se puede asignar a usuarios o a otros roles. Permite administrar permisos de manera centralizada y simplifica la gestión de seguridad.

VENTAJAS DE USAR ROLES



Seguridad granular y controlada



Administración más sencilla y eficiente



Asignación masiva de permisos a múltiples usuarios



Mejores prácticas de seguridad y auditoría



Escalabilidad en entornos grandes y complejos

ROLES FIJOS DE SQL SERVER (EJEMPLOS)

Rol fijo	Descripción
sysadmin	Control total sobre la instancia SQL Server. Acceso ilimitado a todas las funciones.
securityadmin	Administra inicios de sesión, roles y permisos.
serveradmin	Configura la instancia del servidor.
setupadmin	Administra configuración de enlaces y servidores vinculados.
db_owner	Dueño de la base de datos (control total en la base de datos).
db_datareader	Permiso para leer todos los datos de la base de datos.
db_datawriter	Permiso para agregar, modificar y eliminar datos en la base de datos.
db_ddladmin	Permiso para ejecutar comandos DDL (CREATE, ALTER, DROP, etc.).
public	Rol por defecto. Todos los usuarios pertenecen a este rol.

CREACIÓN DE UN ROL PERSONALIZADO (EJEMPLO)

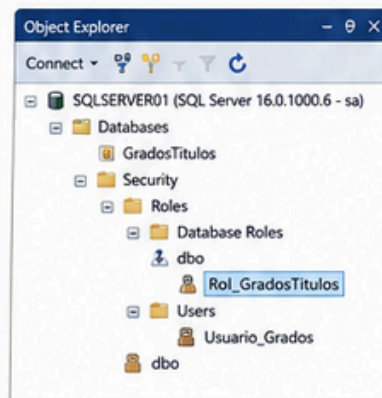


Ejemplo: Crear un rol personalizado

```
-- 1. Crear el rol
CREATE ROLE Rol_GradosTitulos;
GO

-- 2. Asignar permisos al rol
GRANT SELECT, INSERT, UPDATE ON dbo.GradosTitulos
TO Rol_GradosTitulos;
GRANT EXECUTE ON dbo.sp_RegistrarGrado TO Rol_GradosTitulos;
GO

-- 3. Agregar usuarios al rol
EXEC sp_addrolemember 'Rol_GradosTitulos', 'Usuario_Grados';
GO
```



CREAR ROL PERSONALIZADO DESDE SQL SERVER MANAGEMENT STUDIO (SSMS)

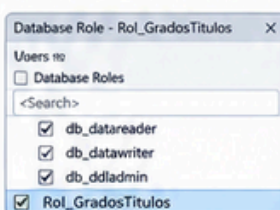
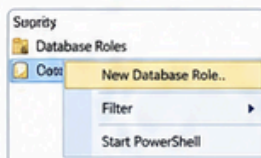
1 Clic derecho en Database Roles > New Database Role...

2 Nombre del rol y descripción (opcional)

3 Asignar permisos en la página Membership

4 Agregar usuarios al rol en la página Members

5 Aceptar y verificar



EJEMPLO DE USO POR ESCENARIO

Escenario	Rol recomendado	Permisos asignados	Usuarios asignados (ejemplo)
Consulta de información	Rol_Consulta	db_datareader	Usuario_Consulta, Usuario_Reportes
Gestión de datos	Rol_GestionDatos	db_datareader, db_datawriter	Usuario_Registro, Usuario_Edicion
Administración de estructura	Rol_AdminEstructura	db_ddladmin	Usuario_Desarrollador
Administración completa	db_owner (solo cuando sea necesario)	Todos los permisos a nivel de BD	Usuario_AdminBD

BUENAS PRÁCTICAS



Asignar solo los permisos necesarios. Principio de mínimo privilegio.



Usar roles en lugar de asignar permisos individualmente.



Revisar y auditar los permisos periódicamente.



Evitar usar sysadmin, asignarlo solo cuando sea imprescindible.



Documentar los roles y su propósito.

CONTROL DE ACCESO MEDIANTE GRANT, DENY Y REVOKE

Administra permisos de manera segura y flexible en SQL Server



¿QUÉ ES EL CONTROL DE ACCESO?

Permite autorizar o restringir acciones sobre objetos de la base de datos (tablas, vistas, procedimientos, etc.) a usuarios o roles.



JERARQUÍA DE PERMISOS EN SQL SERVER



Los permisos se pueden aplicar en diferentes niveles. Un permiso en un nivel superior también puede afectar a los niveles inferiores.

IMPORTANTE
DENY tiene prioridad sobre GRANT.

GRANT: CONCEDER PERMISOS

Permite otorgar permisos a un usuario o rol para que pueda realizar acciones sobre un objeto.



SINTAXIS

```
GRANT permiso [, permiso ...]
ON objeto
TO usuario_o_rol;
```

EJEMPLO

```
GRANT SELECT, INSERT
ON dbo.Alumnos
TO Usuario_Grados;
```

 El usuario o rol podrá consultar e insertar datos en la tabla Alumnos.

DENY: NEGAR PERMISOS

Impide explícitamente que un usuario o rol ejecute una acción, incluso si tiene permisos otorgados por rol o herencia.



SINTAXIS

```
DENY permiso [, permiso ...]
ON objeto
TO usuario_o_rol;
```

EJEMPLO

```
DENY DELETE
ON dbo.Alumnos
TO Usuario_Grados;
```

 El usuario o rol NO podrá eliminar datos de la tabla Alumnos, aunque tenga otros permisos.

REVOKE: REVOCAR PERMISOS

Quita permisos previamente otorgados con GRANT o asignados a través de un rol.



SINTAXIS

```
REVOKE permiso [, permiso ...]
ON objeto
FROM usuario_o_rol;
```

EJEMPLO

```
REVOKE INSERT
ON dbo.Alumnos
FROM Usuario_Grados;
```

 Se elimina el permiso de insertar datos en la tabla Alumnos para el usuario o rol especificado.

COMPARACIÓN ENTRE GRANT, DENY Y REVOKE

ACCIÓN	¿QUÉ HACE?	EFEECTO	EJEMPLO VISUAL
GRANT	Concede permiso para realizar una acción.	Permite el acceso.	 → GRANT SELECT →  →  Puede consultar
DENY	Niega permiso de manera explícita.	Bloquea el acceso, aunque exista un GRANT.	 → DENY SELECT →  →  No puede consultar
REVOKE	Quita un permiso previamente concedido.	Elimina el acceso concedido.	 → GRANT SELECT → REVOKE SELECT →  Permiso eliminado

EJEMPLO COMPLETO EN ESCENARIO

- GRANT**
Se otorga permiso SELECT a un usuario.
- DENY**
Se niega permiso DELETE al usuario.
- REVOKE**
Se revoca permiso INSERT al usuario.

Usuario_Grados

Tabla: dbo.Alumnos	
SELECT	✓
INSERT	✗
UPDATE	✓
DELETE	⊘

Estado final de permisos

- Puede consultar datos.
- No puede insertar datos.
- Puede actualizar datos.
- No puede eliminar datos.

RESULTADO EN SQL SERVER MANAGEMENT STUDIO (SSMS)

Permisos efectivos del usuario

- Seguridad
 - Usuarios
 - Usuario_Grados
 - Permisos
 - dbo.Alumnos
 - Permisos
 - SELECT Concedido
 - INSERT Revocado
 - UPDATE Concedido
 - DELETE Denegado

Interpretación

- ✓ Puede ejecutar SELECT y UPDATE.
- ✗ No puede ejecutar INSERT (porque fue REVOKE).
- ⊘ No puede ejecutar DELETE (porque fue DENY).
- i Los permisos finales son la combinación de GRANT, DENY y REVOKE.

BUENAS PRÁCTICAS

 Usa ROLES en lugar de usuarios individuales para simplificar la administración.

 Aplica el principio de mínimo privilegio: otorga solo los permisos necesarios.

 Revisa y audita permisos periódicamente para mantener la seguridad.

 Evita usar DENY a menos que sea estrictamente necesario.

 Documenta los permisos y su propósito para facilitar el mantenimiento.

CIFRADO Y PROTECCIÓN DE DATOS (TDE, ALWAYS ENCRYPTED)

Protege la información confidencial en reposo, en uso y en tránsito.



TIPOS DE CIFRADO EN SQL SERVER

TDE (Transparent Data Encryption)

Cifra los datos en reposo (archivos de datos, backups y logs).



¿QUÉ PROTEGE?



CARACTERÍSTICAS CLAVE

- ✓ Cifrado automático y transparente para las aplicaciones.
- ✓ Protege contra accesos no autorizados a archivos del sistema.
- ✓ Rendimiento con bajo impacto.
- ✓ Administración centralizada de claves.

ALWAYS ENCRYPTED

Cifra los datos en uso (en la aplicación y en la base de datos).



¿QUÉ PROTEGE?



CARACTERÍSTICAS CLAVE

- ✓ Los datos están cifrados dentro y fuera de la base de datos.
- ✓ Las claves nunca salen del cliente.
- ✓ Protege contra amenazas internas (DBA, administradores).
- ✓ Requiere cambios en la aplicación para manejar claves.

ADMINISTRACIÓN DE CLAVES

TDE: Jerarquía de claves



Always Encrypted: Modelo de claves



COMPARACIÓN TDE vs ALWAYS ENCRYPTED

ASPECTO	TDE (Transparent Data Encryption)	ALWAYS ENCRYPTED
Tipo de protección	 Datos en reposo	 Datos en uso y en reposo
Dónde cifra	 En el motor de base de datos (SQL Server)	 En el cliente (aplicación)
Quién puede ver los datos	 DBA y administradores pueden ver los datos	 Solo la aplicación con la clave puede ver los datos
Granularidad	 A nivel de base de datos	 A nivel de columna
Impacto en la aplicación	 Ninguno	 Requiere cambios en la aplicación
Rendimiento	 Bajo impacto	 Puede tener mayor impacto
Casos de uso	 Cumplimiento normativo, protección de archivos	 Protección de datos sensibles (PII, financieros, salud)

BUENAS PRÁCTICAS



AUDITORÍA Y MONITOREO DE EVENTOS CON SQL SERVER AUDIT

Supervisa, registra y analiza actividades para proteger
y controlar tu base de datos.



¿QUÉ ES SQL SERVER AUDIT?

SQL Server Audit es una funcionalidad nativa que permite registrar eventos y acciones realizadas en el servidor y en la base de datos para fines de seguridad, cumplimiento y diagnóstico.



Proporciona un rastro confiable de actividades para detectar, investigar y responder a incidentes.

¿CÓMO FUNCIONA?

1. ACCIÓN



Usuario o aplicación realiza una acción en SQL Server.

2. AUDITORÍA



SQL Server Audit captura el evento según las reglas definidas.

3. REGISTRO



El evento se escribe en un destino (archivo o Windows Application Log).

4. ANÁLISIS



Los registros se pueden revisar, analizar y reportar.

COMPONENTES PRINCIPALES DE SQL SERVER AUDIT



1. AUDIT (Especifica qué auditar)

Define las acciones y eventos que serán auditados.

Ejemplos de acciones:

- Inicio y cierre de sesión
- Cambios de permisos
- Consultas SELECT/INSERT/UPDATE/DELETE
- Cambios en objetos (CREATE, ALTER, DROP)
- Ejecución de procedimientos almacenados
- Errores y fallos de inicio de sesión
- ...y más



2. AUDIT SPECIFICATION (Dónde y quién)

Asocia el Audit con bases de datos, objetos o servidores y define a qué usuarios o roles aplica.

Se puede aplicar a:



Toda la instancia



Bases de datos específicas



Objetos específicos



3. AUDIT DESTINATION (Dónde se guarda)

Define el destino donde se almacenarán los registros de auditoría.

Tipos de destino:



Archivo (.sqlaudit)



Windows Application Log



Azure Storage (Blob)

ARQUITECTURA DE SQL SERVER AUDIT



EJEMPLOS DE EVENTOS QUE PUEDES AUDITAR

CATEGORÍA	EJEMPLOS DE EVENTOS	DESCRIPCIÓN	BENEFICIO
Seguridad	LOGIN, LOGOUT, FAILED_LOGIN_GROUP	Inicio/cierre de sesión y fallos de autenticación.	Detectar accesos no autorizados.
Permisos	GRANT_PERMISSION, REVOKE_PERMISSION	Cambios en permisos y roles.	Controlar cambios de seguridad.
Datos	SELECT, INSERT, UPDATE, DELETE	Acciones sobre datos sensibles.	Detectar accesos o modificaciones inusuales.
Esquemas/Objetos	CREATE, ALTER, DROP (DATABASE, TABLE, VIEW, etc.)	Cambios en la estructura de la base de datos.	Prevenir cambios no autorizados.
Ejecuciones	EXECUTE (Stored Procedures, Functions, etc.)	Ejecución de código y procedimientos.	Monitorear uso de objetos críticos.
Errores	ERRORLOG, AUDIT_FAILED, LOGIN_ATTEMPT	Errores críticos y eventos de auditoría fallidos.	Identificar problemas y ataques.

PASOS PARA IMPLEMENTAR SQL SERVER AUDIT

1. Crear el Audit

```
CREATE SERVER AUDIT MiAudit  
TO FILE (FILEPATH = 'D:\Auditoria');
```



2. Crear Audit Specification

```
CREATE SERVER AUDIT SPECIFICATION MiAuditSpec  
FOR SERVER AUDIT MiAudit  
ADD (SELECT, INSERT, UPDATE, DELETE ON DATABASE::MiDB),  
ADD (FAILED_LOGIN_GROUP)  
WITH (STATE = ON);
```



3. Habilitar el Audit

```
ALTER SERVER AUDIT MiAudit WITH (STATE = ON);
```



4. Consultar los registros

```
SELECT * FROM sys.fn_get_audit_file  
( 'D:\Auditoria\*.sqlaudit', DEFAULT, DEFAULT );
```



EJEMPLO VISUAL DE UN REGISTRO DE AUDITORÍA

event_time	action_id	session_id	server_principal_name	database_name	statement	result
2024-05-14 10:21:33	LGIS	55	Usuario_Grados	MiDB	Login succeeded	SUCCESS
2024-05-14 10:22:01	SLCT	55	Usuario_Grados	MiDB	SELECT * FROM Alumnos	SUCCESS
2024-05-14 10:25:10	UPDT	55	Usuario_Grados	MiDB	UPDATE Alumnos SET Nombre='Ana'	SUCCESS
2024-05-14 10:28:45	LGIF	63	HackerUser	NULL	Login failed	FAILED

action_id:
Tipo de acción realizada.

session_id:
Sesión que ejecutó la acción.

server_principal_name:
Usuario que realizó la acción.

result:
Resultado de la acción.

BUENAS PRÁCTICAS



Audita solo lo necesario para reducir impacto en rendimiento.



Combina SQL Server Audit con otras herramientas de monitoreo y SIEM.



Almacena los archivos en ubicaciones seguras y con respaldo.



Protege los archivos de auditoría; contienen información sensible.



Revisa los registros periódicamente y configura alertas.



Documenta y prueba tus políticas de auditoría regularmente.